PROJECT PLAN

Smart – Attendance

University of Eswatini

Practices In Software Engineering – CSC392

1. Introduction.

The Smart Attendance System is a software–hardware integrated solution designed to automate the process of recording student attendance using barcode scanning technology.

Traditionally, attendance in academic institutions is recorded manually using paper-based methods, which are time-consuming, error-prone, and susceptible to manipulation such as proxy attendance. This project aims to address these limitations by introducing a digital system that improves accuracy, efficiency, and data management.

Aims and Objectives

2. To automate the attendance recording process
3. To eliminate proxy attendance and improve data integrity
4. To provide real-time attendance tracking and reporting
5. To integrate hardware (barcode scanning) with software systems

Names and Responsibilities

Wakhile Mabuza – Team Manager

Sean Rodrigues – Architect

Njabulo Simelane – Designer

Sakhiwo Magongo – Programmer.

2. User / Client Involvement

Primary Client

The primary users of the system are lecturers and students.

- Lecturers will provide system requirements such as attendance rules and reporting needs
- Students will participate in system testing and usability feedback

Resources Provided by Users

- Student ID cards (with barcodes)
- Class schedules and course data

Timing of Involvement

- Requirements phase: input on system needs
- Testing phase: usability and functionality feedback

3. Risks

Potential risks have been identified early in the project so that appropriate strategies can be put in place before they cause problems. The following risks are considered the most significant for this project.

Risk	Description	Mitigation Strategy
Hardware failure	Scanner or camera may fail	Use alternative device (phone/webcam)
Integration challenges	Hardware–software mismatch	Early prototyping and testing
Time constraints	Limited semester duration	Agile development with milestones
Team coordination	Uneven contribution	Regular meetings and GitHub tracking
Data errors	Incorrect scans or duplicates	Validation checks and error handling

4. Standards, guidelines and procedures

To ensure consistency and quality:

- **Version Control:** Git and GitHub will be used for collaboration
- **Coding Standards:** Consistent naming conventions and modular design
- **Documentation:** All features and changes will be documented
- **Meeting Protocols:** Weekly meetings with recorded minutes
- **Development Methodology:** Agile (iterative development approach)

5. Organization of the project.

Wakhile Mabuza – Team Manager
Responsible for overall coordination and management of the project.
Duties include:

- Planning and scheduling project activities
- Monitoring team progress and ensuring deadlines are met
- Managing the GitHub repository and approving contributions
- Facilitating team meetings and communication
- Ensuring proper documentation and submission

Sean Rodrigues – System Architect
Responsible for the overall system design and technical structure.
Duties include:

- Designing system architecture (software and hardware integration)
- Defining system components and interactions
- Developing UML diagrams (use case, sequence, architecture)
- Ensuring system scalability and maintainability

Njabulo Simelane - UI/UX Designer
Responsible for designing the user interface and user experience.
Duties include:

- Creating wireframes and interface layouts
- Designing intuitive and user-friendly interfaces
- Ensuring consistency in design across the system
- Collaborating with developers during implementation

Sakhiwo Magongo – Programmer (Backend & Integration)
Responsible for system implementation and integration.
Duties include:

- Developing backend logic and APIs
- Implementing barcode scanning functionality
- Managing database interactions
- Integrating frontend and backend components

5.2 Team Coordination and Communication

The team will collaborate using GitHub as the primary version control and collaboration platform. All changes to the system will be made through commits and pull requests, which will be reviewed and approved by the Team Manager.

Regular meetings will be held to:

- Track progress
- Address challenges
- Make key decisions

Communication will be maintained through WhatsApp and scheduled meetings.

5.3 Training Requirements

To ensure all team members are adequately prepared, the following training areas have been identified:

- Use of Git and GitHub for version control
- Understanding of barcode scanning technology
- Familiarity with the development stack (React, Node.js, PostgreSQL)

Training will take place during the early stages of the project (Week 1–2).

5.4 Project Dependencies

The successful execution of the project depends on:

- Availability of barcode scanning hardware (or webcam alternative)
- Stable internet connectivity for collaboration
- Proper coordination among team members

6. Project Phases

The project will follow an **Agile development methodology**, divided into six (6) weeks, with each week representing a mini-iteration focusing on specific deliverables.

6.1 Development Phases and Activities

Week 1: Requirements & Planning

- Finalize project idea
- Gather system requirements
- Define system scope
- Set up GitHub repository

Responsible:

- Team Manager (lead coordination)
- Architect (requirements structuring)
- All members contribute

Week 2: System Design

- Design system architecture
- Create UML diagrams
- Design database structure
- Plan UI layout

Responsible:

- Architect (lead design)
- Designer (UI design)
- Team Manager (coordination)

Week 3: Initial Implementation

- Set up development environment
- Implement authentication module
- Start attendance session feature

Responsible:

- Programmer (lead development)
- Architect (support system structure)

Week 4: Core System Development

- Implement barcode scanning
- Connect frontend and backend
- Store attendance data

Responsible:

- Programmer (core implementation)
- Designer (UI integration)

Week 5: Integration & Testing

- Integrate all modules
- Perform unit and integration testing
- Identify and fix bugs **Responsible:**
- Programmer (bug fixing)
- Team Manager (testing coordination)
- All members participate in testing

Week 6: Finalization & Submission

- Perform system testing
- Finalize documentation
- Prepare for submission and demonstration

Responsible:

- Team Manager (final coordination)
- All members contribute.

6.2 Milestones

Milestone	Week	Deliverable
Requirements Completed	Week 1	Approved system idea
System Design Completed	Week 2	UML diagrams & architecture
Prototype Ready	Week 3	Basic working system
Core Features Completed	Week 4	Functional attendance system
System Tested	Week 5	Integrated and tested system
Final Submission	Week 6	Completed project

6.3 Critical Path

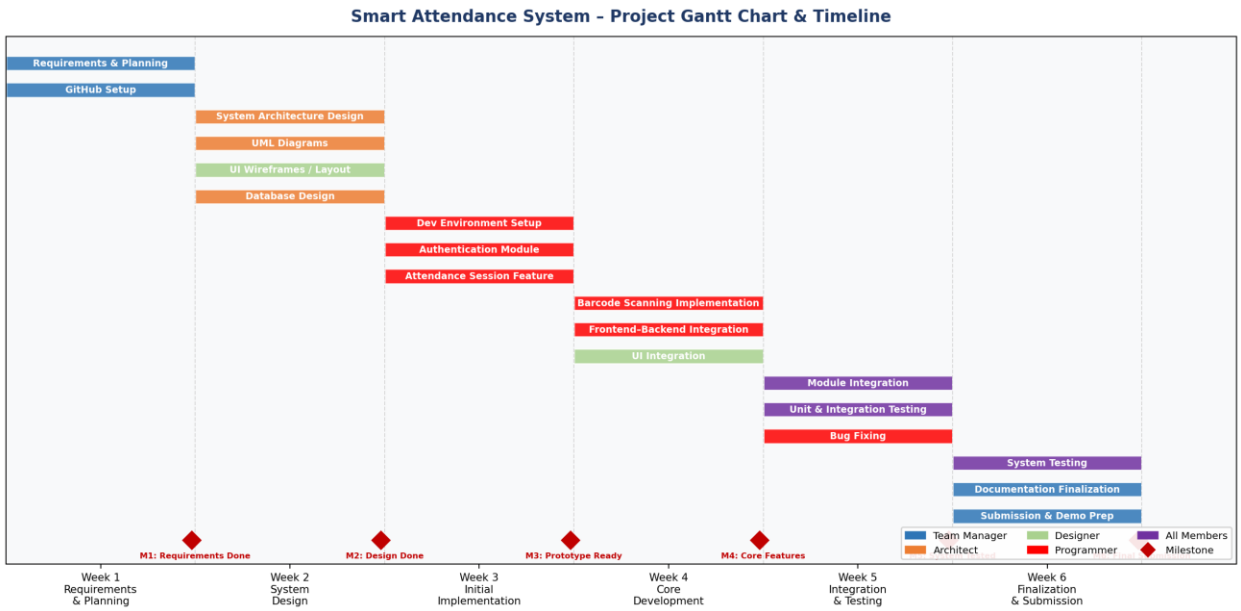
The critical path includes:

- Requirements → Design → Implementation → Testing → Finalization

Any delay in these activities will directly affect the project timeline.

6.4 Gantt Chart & Visual Timeline

The Gantt chart below illustrates the full project timeline across six weeks, showing task distribution by team member role and key milestones.



Week	Team Manager	Architect	Designer	Programmer
Week 1	Coordinate planning	Define requirements	Assist	Assist
Week 2	Oversee design	Lead architecture	Design UI	Assist
Week 3	Monitor progress	Support structure	Assist UI	Develop backend
Week 4	Coordinate	Support integration	UI integration	Core development
Week 5	Lead testing	Assist testing	UI fixes	Debug & fix
Week 6	Final submission	Review system	Final UI polish	Final fixes

7. Requirements Analysis and Design

Functional Requirements

- Lecturer authentication
- Attendance session creation
- Barcode scanning for student identification
- Attendance recording and storage
- Report generation

Non-Functional Requirements

- System must be fast and responsive
- High reliability and accuracy
- Easy-to-use interface

Design Tools

- UML diagrams (use case, sequence diagrams)
- System architecture diagrams

8. Implementation

Technologies

- Frontend: React
- Backend: Node.js + Express
- Database: PostgreSQL
- Hardware: Barcode scanner or webcam

Development Approach

- Modular implementation (authentication, attendance, reporting)
- Continuous integration using GitHub.

9. Testing

Testing Strategy

- Unit testing (individual components)
- Integration testing (combined modules)
- System testing (full application)

Test Environment

- Local machines and web browsers
- Barcode scanning devices

Testing Procedure

1. Test individual modules
2. Integrate modules step-by-step
3. Perform system-wide testing
4. Fix bugs and retest

10. RESOURCES

Hardware

- Laptops
- Barcode scanner / webcam

Software

- GitHub
- Visual Studio Code
- Web browsers

Human Resources

- Development team (4 members)

11. QUALITY ASSURANCE

To ensure the system meets required standards:

- Code reviews before merging changes
- Use of GitHub for version tracking
- Continuous testing throughout development
- Bug tracking and resolution
- Proper documentation of all modules

12. CHANGES

Changes are inevitable during the software development process. To manage changes effectively, the team will follow a structured change management process.

Change Request Process:
Any proposed change must be communicated to the team leader and discussed with the team before implementation.

- **Approval of Changes:**
  The team leader will review and approve all changes before they are applied to the main project.

- **Version Control:**
  All changes will be implemented using GitHub through commits and pull requests to ensure traceability.

- **Impact Analysis:**
  Before implementing any change, its impact on the project timeline, functionality, and other components will be evaluated.

- **Documentation of Changes:**
  All changes will be recorded to maintain a clear history of the project's development.

This structured approach ensures that changes are handled in an organized manner without negatively affecting the project's progress.